



Universidad Autónoma de Guadalajara Campus Tabasco

Departamento Académico de
Ingenierías

[Tipo de Actividad: Ensayo, Resumen, etc.]

Read me

Programa

Ing. En Software y minería de datos

Asignatura

PROGRAMACION ORIENTADA A OBJETOS

Presenta

Paulina Cancino Vichi

Ary Gabriela de la Cruz Peralta

Camila Andrea Jaimez Martinez

Sofia Watts Feria

Docente

MANUEL BALDERAS

Villahermosa, Tabasco, México

17 de septiembre del 2026



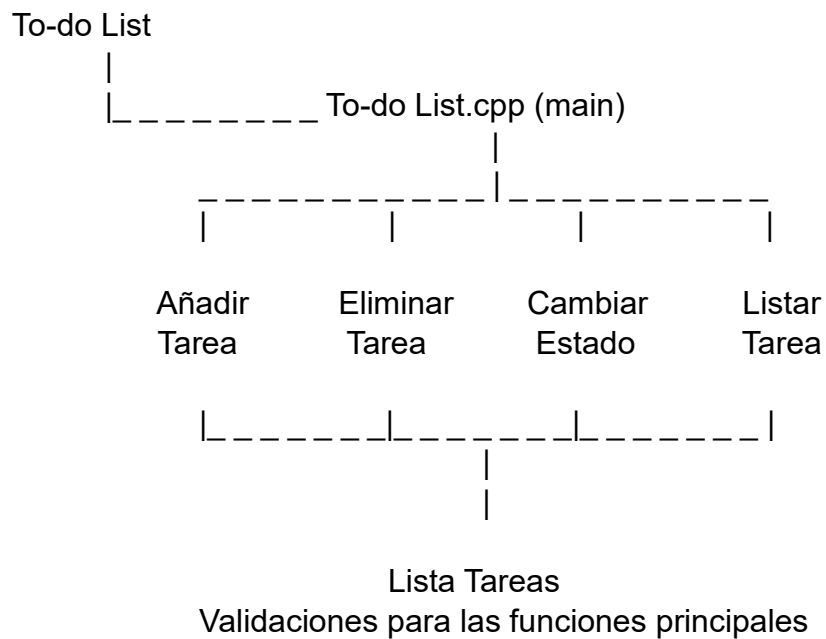
TO-DO LIST

Se desarrolló un programa en C++ donde se puedan guardar las tareas del usuario.

Operaciones Principales:

- Añadir una tarea.
 - Nombre
 - Descripción
 - Fecha
 - Estado (Pendiente, En Progreso, Completado)
- Listar las tareas registradas
- Cambiar el estado de una tarea
- Eliminar una tarea

1. JERARQUÍA GENERAL DEL PROGRAMA



2. CLASES Y FUNCIONES

2.1 Clase “Tarea”

Archivos: Tarea.h y Tarea.cpp

Hecho por: Camila

Es la base del programa porque representa una sola tarea.

Cada objeto “Tarea” contiene:

- ID.
- Título.
- Descripción.
- Estado.
- Fecha.

También contiene los métodos necesarios para consultar y modificar sus datos.

Por ejemplo, cuando se cambia el estado de una tarea, finalmente es el objeto Tarea el que modifica su atributo estado.

También se encarga del sistema de IDs automáticos

2.2 Función “Añadir Tarea”

Archivos: AnadirTarea.h y AnadirTarea.cpp

Hecho por: Camila

Esta parte funciona como la interfaz para crear una nueva tarea.

Flujo:

1. El usuario selecciona “Añadir tarea” en To-do list.cpp
2. AnadirTarea.cpp solicita los datos de la nueva tarea
3. Validaciones.cpp comprueba que los datos sean válidos
4. Tarea.cpp crea el objeto Tarea y le asigna un ID
5. ListaTareas.cpp recibe la tarea y la guarda en el vector<Tarea>
6. El programa regresa al menú principal

2.3 Clase “ListaTareas”

Archivos: ListaTareas.h y ListaTareas.cpp

Base realizada por: Ary

Función de listado integrada por: Paulina

Conexión para cambio de estado: integración con el trabajo de Sofía

ListaTareas administra el conjunto completo de tareas

Mientras que:

Tarea = una tarea.

ListaTareas = todas las tareas.

Internamente contiene el vector<Tarea> donde se almacenan los objetos.

Operaciones que realiza la clase:

- Agregar una tarea
- Buscar una tarea por ID
- Comprobar si una tarea existe
- Eliminar una tarea
- Obtener la cantidad de tareas
- Mostrar un resumen
- Listar todas las tareas
- Conectar el cambio de estado
- Renumerar los IDs después de una eliminación

Esto permite mantener el vector como un atributo privado y acceder a las tareas mediante métodos públicos.

2.4 Función “ListarTarea”

Ubicación: ListaTareas.cpp

Hecho por: Paulina

Aunque la función está físicamente dentro del archivo de ListaTareas, corresponde a la parte de Paulina.

Se encarga de mostrar las tareas almacenadas en forma de tabla

ID	Descripción	Estado	Fecha
1	Hacer tarea	pendiente	17/09/2026
2	Estudiar C++	en progreso	17/09/2026
3	Terminar proyecto	completada	16/09/2026

También verifica si la lista está vacía antes de intentar mostrarla.
La conexión con el menú se realiza mediante `listarTareasUI()` en el programa principal.

2.5 Función “CambiarEstado”

Archivos: CambiarEstado.h y CambiarEstado.cpp

Hecho por: Sofía

Esta parte permite seleccionar una tarea y cambiar su estado.

Estados:

- Pendiente
- En progreso
- Completada

Flujo:

1. El usuario selecciona "Cambiar estado"
2. CambiarEstado.cpp solicita el ID y el nuevo estado
3. ListaTareas.cpp busca la tarea correspondiente por su ID
4. Tarea.cpp ejecuta modificar_estado() y actualiza el estado
5. El programa regresa al menú principal

2.6 Función "EliminarTarea"

Ubicación: ListaTareas.cpp

Hecho por: Ary

Esta parte permite eliminar una tarea mediante su ID.

Flujo:

1. El usuario selecciona "Eliminar tarea"
2. ListaTareas.cpp muestra las tareas y solicita el ID
3. Validaciones.cpp valida el ID y solicita confirmación
4. ListaTareas.cpp elimina la tarea del vector<Tarea>
5. Tarea.cpp permite reenumerar los IDs y actualiza el siguiente ID disponible
6. El programa regresa al menú principal

2.7 Validaciones

Archivos: Validaciones.h y Validaciones.cpp

Hecho por: Ary

Este archivo funciona como apoyo para las diferentes partes del programa.

Se encarga de validar cosas como:

- Que se ingrese un número cuando se espera un número.
- Que el número esté dentro de un rango
- Que un texto no esté vacío
- Que el texto no exceda cierta longitud
- Confirmaciones mediante s/n

Por eso las validaciones no son una opción independiente del menú. Son herramientas utilizadas por las demás funciones.

2.8 Menu

Archivos: menu.h y menu.cpp

Hecho por: Paulina

El menú presenta al usuario las opciones disponibles y recibe la selección

```
===== TO-DO LIST =====  
1. Añadir tarea  
2. Listar tareas  
3. Cambiar estado  
4. Eliminar tarea  
5. Salir  
=====  
Selecciona una opcion:
```

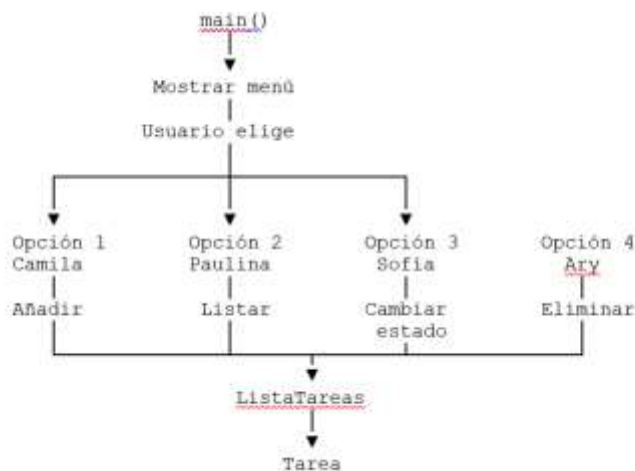
El menú se encarga de mostrar y recibir la opción, mientras que main() decide qué función ejecutar según esa opción

2.9 Main

Archivo: To-do list.cpp

Hecho por: Paulina / integración del trabajo del equipo

Este archivo contiene el único main() del proyecto
Es el punto central que conecta todos los módulos



3. ESTRUCTURA COMPLETA DEL PROGRAMA



4. ORGANIZACIÓN DEL EQUIPO

Camila	estructura de Tarea y funcionalidad para añadir tareas
Ary	administración de ListaTareas, eliminación y validaciones
Sofia	funcionalidad para cambiar el estado de las tareas
Paulina	menú, listado de tareas e integración de las diferentes funciones con el programa principal

5. CORRECCIONES

5.1 Archivos duplicados de Tarea

Problema: Existían dos versiones diferentes de la clase Tarea: la de Camila y otra utilizada en la parte de Paulina. Tenían diferentes atributos, constructores y nombres de funciones, lo que provocaba incompatibilidad

Solución: Se tomó como clase principal la Tarea de Camila y se adaptaron las demás partes del programa para trabajar con esa misma clase

5.2 Archivos duplicados de ListaTareas

Problema: Ary y Paulina tenían versiones diferentes de ListaTareas, con métodos y estructuras distintas. Mantener ambas provocaría conflictos al compilar

Solución: Se utilizó como base la estructura de ListaTareas de Ary y se integró dentro de ella la función listarTareas() desarrollada por Paulina

5.3 Archivos duplicados de validaciones

Problema: Existían dos sistemas diferentes de validación. Uno trabajaba directamente con un vector<Tarea>, mientras que el vector oficial estaba encapsulado dentro de ListaTareas

Solución: Se conservaron las validaciones de Ary como sistema general del programa y se eliminaron las validaciones duplicadas que dependían directamente del vector

5.4 Descripción de la tarea no se guardaba correctamente

Problema: En el constructor de Tarea, Camila recibía la descripción mediante el parámetro desc, pero se intentaba asignar otra variable llamada descripcion

Solución: Se corrigió la asignación para que el atributo descripcion reciba correctamente el valor de desc.

5.5 El contador de IDs no estaba inicializado

Problema: En Tarea.h existía static int sig_id, pero faltaba inicializarlo en Tarea.cpp, lo que podía provocar un error de enlace al compilar

Solución: Se inicializó sig_id comenzando desde 1.

5.6 El constructor recibía un ID pero no lo manejaba correctamente

Problema: El constructor de Tarea recibía un parámetro id, pero originalmente el programa generaba otro mediante sig_id, haciendo confuso el funcionamiento

Solución: Se estableció que cuando se crea una tarea nueva con ID 0, el sistema genera automáticamente el ID utilizando sig_id

5.7 IDs desordenados después de eliminar una tarea

Problema: Si existían las tareas 1, 2, 3 y 4 y se eliminaba la número 2, las restantes quedaban con IDs 1, 3 y 4

Solución: Después de eliminar una tarea, se recorren las tareas restantes y se vuelven a asignar los IDs desde 1 para mantener una numeración consecutiva

5.8 El siguiente ID seguía utilizando la numeración anterior

Problema: Aunque se renumeraran las tareas después de eliminar, sig_id podía conservar el valor anterior. Por ejemplo, después de quedar 1, 2 y 3, una nueva tarea podía recibir el ID 5

Solución: Se añadió una función para actualizar sig_id después de eliminar. Ahora el siguiente ID corresponde a la cantidad de tareas existentes más 1

5.9 No existía una forma de modificar el ID

Problema: El atributo id de Tarea es privado, por lo que ListaTareas no podía cambiarlo directamente durante la renumeración.

Solución: Se añadió modificar_id() a la clase Tarea, manteniendo el encapsulamiento.

5.10 Código de cambio de estado utilizaba otra estructura de Tarea

Problema: La parte original de Sofía tenía su propio struct Tarea, mientras que el proyecto ya utilizaba la clase Tarea de Camila

Solución: Se adaptó el código de Sofía para trabajar con la clase oficial Tarea a través de ListaTareas

5.11 Sofía tenía su propio vector de tareas

Problema: El código de cambio de estado utilizaba un vector global independiente. Esto significaba que podía modificar tareas distintas a las almacenadas en la lista principal

Solución: Se eliminó la dependencia del vector independiente y el cambio de estado ahora trabaja directamente con el objeto ListaTareas utilizado por todo el programa

5.12 Había más de un main()

Problema: Algunas partes desarrolladas individualmente tenían su propio main(). Al unirlos, C++ no puede tener varios puntos de entrada en el mismo programa.

Solución: Se dejó únicamente el main() de To-do list.cpp. Las demás partes se transformaron en funciones que son llamadas desde ese archivo.

5.13 Cambio de estado no estaba conectado con ListaTareas

Problema: La funcionalidad independiente de Sofía no tenía una manera de modificar una tarea almacenada dentro del vector privado de ListaTareas

Solución: Se añadió una función de conexión en ListaTareas que busca una tarea mediante su ID y llama al método correspondiente de Tarea para modificar su estado

5.14 Añadir tarea no estaba adaptado a la arquitectura final

Problema: La versión anterior para añadir una tarea solo manejaba parte de la información y utilizaba métodos pertenecientes a una versión diferente de ListaTareas

Solución: Se creó AnadirTarea.cpp/.h, utilizando la clase Tarea de Camila, las validaciones de Ary y la función oficial de ListaTareas para almacenar la tarea

5.15 Las funciones de cada integrante estaban mezcladas en el programa principal

Problema: Si toda la lógica se colocaba directamente dentro de main(), era más difícil identificar el trabajo de cada integrante y mantener el código organizado.

Solución: Se mantuvieron archivos independientes para añadir tareas, cambiar estado, menú y las clases principales. main() únicamente conecta las opciones del menú con esas funciones.

5.16 La opción 2 del menú no estaba conectada con el listado

Problema: La función de Paulina para mostrar las tareas existía, pero debía conectarse con la opción correspondiente del menú.

Solución: Se añadió listarTareasUI(), que conecta la opción 2 con lista.listarTareas().

5.17 El listado utilizaba posición en lugar del ID real

Problema: El listado podía mostrar $i + 1$, es decir, la posición dentro del vector, mientras que otras funciones como eliminar trabajaban con el atributo id

Solución: Se modificó el listado para mostrar obtener_id(), de modo que todo el programa utilice el mismo identificador

5.18 Lista vacía mostraba información innecesaria

Problema: Al intentar listar tareas cuando no existía ninguna, podía mostrarse una tabla vacía

Solución: Se añadió una validación para mostrar el mensaje "No hay tareas registradas en la lista." y terminar la función

5.19 El listado no tenía un formato visual claro

Problema: La información de las tareas podía resultar difícil de identificar en

consola

Solución: Se organizó la salida en una tabla con las columnas ID, Descripción, Estado y Fecha

5.20 Eliminar podía recibir un ID inexistente

Problema: El usuario podía intentar eliminar una tarea que no existía

Solución: Antes de eliminar se verifica la existencia del ID y se muestra un mensaje cuando no corresponde a ninguna tarea

5.21 Una tarea podía eliminarse accidentalmente

Problema: Seleccionar un ID podía eliminar inmediatamente la tarea

Solución: Se mantuvo una confirmación s/n antes de realizar definitivamente la eliminación

5.22 Entradas numéricas inválidas

Problema: El usuario podía introducir letras, valores fuera de rango u otros datos cuando el programa esperaba un número.

Solución: Se utilizaron las funciones de Validaciones.cpp de Ary para comprobar que la entrada sea un número válido y esté dentro del rango permitido

5.23 Campos de texto vacíos

Problema: El usuario podía intentar crear una tarea sin escribir información en campos obligatorios

Solución: Se utilizó leerTextoNoVacio() para volver a solicitar la información cuando el texto está vacío

5.24 Problema al escribir el título de la primera tarea

Problema: Después de seleccionar la opción 1 mediante cin >>, el Enter quedaba almacenado en el buffer. El siguiente getline() lo interpretaba como una entrada vacía y aparecía inmediatamente "Error: el texto no puede estar vacío."

Solución: Se limpió el buffer después de leer la opción del menú utilizando cin.ignore() antes de entrar a las funciones que trabajan con getline().

5.25 Faltaba la librería necesaria para limpiar correctamente el buffer

Problema: Para descartar todo el contenido pendiente hasta el salto de línea se necesitaba numeric_limits

Solución: Se añadió <limits> en To-do list.cpp

5.26 UTF-8 estaba colocado en archivos donde no era necesario

Problema: No era necesario configurar la consola individualmente desde cada archivo .cpp.

Solución: La configuración UTF-8 se dejó únicamente al comienzo del main() en To-do list.cpp, ya que se ejecuta una vez para todo el programa.

5.27 Faltaba conexión entre las opciones del menú y el trabajo de cada integrante

Problema: Las funciones existían de manera independiente, pero todavía no formaban un programa completo

Solución: To-do list.cpp quedó como punto central: opción 1 conecta con añadir tarea de Camila, opción 2 con listar de Paulina, opción 3 con cambiar estado de Sofía, opción 4 con eliminar de Ary y opción 5 finaliza el programa

5.28 Nombres de métodos incompatibles entre las versiones del proyecto

Problema: Había funciones como agregarTarea, eliminarTarea, cambiarEstado y otras versiones con nombres distintos, como agregar_tarea y eliminar_tarea

Solución: Se unificaron las llamadas para utilizar los métodos de la estructura oficial de ListaTareas, agregando únicamente las funciones de conexión necesarias

5.29 El vector de tareas debía permanecer encapsulado

Problema: Algunas versiones del código intentaban validar o modificar directamente el vector<Tarea>, aunque este pertenece a la parte privada de ListaTareas

Solución: Las operaciones se realizan mediante métodos públicos de ListaTareas, respetando el principio de encapsulamiento de POO

6. REPORTE SEMANAL

Semana 1: "Planeación y desarrollo de la estructura inicial"

Durante la primera semana se trabajó en la planeación y construcción de la estructura inicial del proyecto To-Do List, cuyo objetivo fue desarrollar una aplicación de consola en C++ que permitiera administrar diferentes tareas mediante operaciones de creación, consulta, modificación y eliminación.

Inicialmente, se analizaron los requerimientos del proyecto y se identificaron las principales funcionalidades que debía contener el programa. A partir de esto, se distribuyeron las actividades entre las integrantes del equipo con la finalidad de trabajar de manera colaborativa en diferentes módulos del sistema.

Se comenzó con el desarrollo de la clase Tarea, en la cual se definieron los atributos necesarios para representar cada tarea dentro del programa, como el título, la descripción, el estado, la fecha y el identificador. También se desarrollaron los métodos necesarios para obtener y modificar dicha información.

De igual manera, se trabajó en la clase ListaTareas, cuya finalidad fue administrar el conjunto de tareas registradas durante la ejecución del programa. Para almacenar los objetos se utilizó un vector<Tarea>, permitiendo agregar, buscar, consultar y posteriormente eliminar tareas.

Durante esta semana también se desarrolló la estructura inicial del menú principal. Se establecieron cinco opciones: añadir una tarea, listar las tareas registradas, cambiar el estado de una tarea, eliminar una tarea y salir del programa. El menú se separó en sus propios archivos con la finalidad de mantener una mejor organización del código.

Asimismo, se comenzó con el desarrollo del sistema de validaciones para controlar los datos ingresados por el usuario. Se plantearon funciones reutilizables para verificar entradas numéricas, textos vacíos y respuestas de confirmación.

Al finalizar la primera semana se contó con la estructura principal del proyecto y con las clases y archivos necesarios para continuar con el desarrollo de las funcionalidades específicas del programa.

Semana 2: "Implementación de las funcionalidades principales"

Durante la segunda semana se trabajó en la implementación de las principales funcionalidades del sistema, utilizando como base las clases y estructuras desarrolladas durante la primera etapa.

Se desarrolló la funcionalidad para añadir tareas, mediante la cual se solicitaron al usuario los datos necesarios para crear un nuevo objeto de tipo Tarea. Una vez capturada y validada la información, la nueva tarea se almacenó dentro del vector administrado por ListaTareas.

También se trabajó en el manejo de los identificadores de las tareas. Se implementó un contador estático para generar automáticamente un ID diferente para cada nueva tarea registrada, facilitando posteriormente su búsqueda, modificación y eliminación.

Posteriormente, se desarrolló la funcionalidad para listar las tareas registradas. Para mejorar la visualización de la información, se organizó la salida en formato de tabla, mostrando el ID, descripción, estado y fecha correspondiente a cada tarea. También se añadió una validación para detectar cuando la lista se encontraba vacía y mostrar un mensaje informativo en lugar de intentar imprimir una tabla sin elementos.

Se implementó también la funcionalidad para cambiar el estado de una tarea. Para realizar esta operación se utilizó el ID como identificador de búsqueda. Una vez localizada la tarea correspondiente, se permitió modificar su estado utilizando únicamente los valores establecidos por el sistema: pendiente, en progreso y completada.

De igual manera, se desarrolló la funcionalidad para eliminar tareas. Se implementó la búsqueda mediante ID, se mostró un resumen de las tareas disponibles y se solicitó una confirmación antes de realizar la eliminación. Una vez confirmada la operación, la tarea correspondiente se eliminó del vector.

Durante esta etapa también se fortaleció el sistema de validaciones. Se implementaron funciones para comprobar que los textos ingresados no estuvieran vacíos y no excedieran la longitud establecida. Para las entradas numéricas se verificó que el usuario escribiera únicamente números y que estos se encontraran dentro del rango requerido dependiendo de la operación realizada.

Al finalizar la segunda semana se contó con las principales operaciones del sistema desarrolladas de manera independiente: añadir, listar, cambiar estado y eliminar tareas.

Semana 3: "Integración, corrección de errores y pruebas finales"

Durante la tercera semana se trabajó principalmente en la integración de las diferentes partes desarrolladas por las integrantes del equipo. Debido a que las funcionalidades se habían desarrollado inicialmente en diferentes archivos, fue necesario revisar la compatibilidad entre las clases, métodos y estructuras utilizadas.

Se realizó la integración de las diferentes funcionalidades dentro de una única clase `ListaTareas`. Esto permitió que las operaciones de agregar, listar, cambiar estado y eliminar trabajaran sobre el mismo vector `<Tarea>`, evitando la existencia de diferentes listas o estructuras que almacenaran información de manera independiente.

También se integró la función encargada de listar las tareas con la estructura previamente desarrollada para `ListaTareas`. De esta manera se evitó mantener dos versiones diferentes de la misma clase y se consiguió que el listado utilizara directamente las tareas almacenadas en el sistema.

La funcionalidad de cambio de estado también fue adaptada durante la integración. Se eliminó la necesidad de utilizar una estructura `Tarea` adicional y un vector independiente, permitiendo utilizar directamente la clase `Tarea` y la clase `ListaTareas` oficiales del proyecto. Asimismo, se eliminó el `main()` independiente que existía en dicho módulo para mantener un único punto de entrada al programa.

Se realizaron también correcciones relacionadas con los IDs de las tareas. Después de detectar que la eliminación de una tarea podía provocar saltos en la numeración, se añadió un proceso de renumeración. Por ejemplo, si existían las tareas con IDs 1, 2, 3 y 4 y se eliminaba la tarea número 2, los IDs restantes se reorganizaron como 1, 2 y 3. También se actualizó el contador utilizado para generar el siguiente ID, evitando que al añadir una nueva tarea aparecieran números repetidos o saltos en la secuencia.

Durante las pruebas se detectó y corrigió un problema relacionado con el buffer de entrada. Al seleccionar una opción del menú y posteriormente solicitar información mediante `getline()`, el salto de línea restante podía interpretarse como una entrada vacía. Se añadió la limpieza correspondiente del buffer para evitar que el título de la primera tarea fuera detectado automáticamente como vacío.

También se configuró la consola de Windows para trabajar con UTF-8, permitiendo visualizar correctamente caracteres especiales utilizados en los mensajes del programa, como acentos y la letra ñ.

Una vez integradas las funcionalidades, se realizaron diferentes pruebas para comprobar el comportamiento del programa. Se verificó la creación de tareas, la

asignación automática de IDs, la visualización del listado, la modificación de estados, la eliminación de tareas, la reenumeración de identificadores y el funcionamiento de las validaciones ante entradas incorrectas.

Finalmente, se organizó el programa para contar con un único archivo que contuviera la función `main()`, desde el cual se conectaron las diferentes opciones del menú con los módulos correspondientes. Los archivos finales también se integraron en el repositorio colaborativo de GitHub, permitiendo mantener un registro de los avances realizados durante el desarrollo.

Al finalizar la tercera semana se obtuvo una versión funcional e integrada del To-Do List, en la cual las diferentes partes desarrolladas por el equipo lograron comunicarse correctamente dentro de un mismo sistema.

7. CONCLUSIÓN

El desarrollo del To-Do List permitió aplicar los principales conceptos de programación orientada a objetos mediante un programa capaz de agregar, listar, modificar y eliminar tareas. Durante el proyecto fue necesario integrar el trabajo de cada integrante, corregir incompatibilidades y conectar los diferentes archivos para que funcionaran como un solo sistema. Además, se implementaron validaciones y un manejo adecuado de los IDs para mejorar el funcionamiento del programa. En general, este proyecto permitió comprender la importancia de organizar el código, trabajar de manera colaborativa y lograr que diferentes módulos se comuniquen correctamente.

